

Chương 6: Phương pháp tối ưu Gradient Descent

Học máy 67CS1

Mục lục

I	Gradient Descent cho hàm một biến	2
1	Khái niệm cơ bản	2
2	Công thức của Gradient Descent	2
3	Lựa chọn Tốc độ học η	2
4	Thuật toán Gradient Descent	2
5	Ví dụ Minh Họa	3
6	Kết luận	3
6.1	Giải thích mã	4
II	Gradient Descent cho hàm nhiều biến	5
7	Khái niệm cơ bản	5
8	Công thức toán học của Gradient	5
9	Cập nhật tham số	6
10	Quy trình Gradient Descent	6
11	Ví dụ minh họa	6
12	Những thách thức và biến thể	6
13	Kết luận	7
14	Code lập trình	7
14.1	Giải thích	8

Phần I

Gradient Descent cho hàm một biến

1 Khái niệm cơ bản

Gradient Descent (GD) là một phương pháp tối ưu hóa rất phổ biến và hữu ích trong học máy và toán học. Nó được sử dụng để tìm giá trị cực tiểu (hoặc cực đại) của một hàm mục tiêu. Trong bài viết này, chúng ta sẽ tìm hiểu về Gradient Descent cho hàm một biến.

Hãy bắt đầu bằng cách định nghĩa một hàm mục tiêu đơn giản $f(x)$ mà chúng ta muốn tối ưu hóa:

$$f(x)$$

Gradient Descent là một thuật toán lặp đi lặp lại, xuất phát từ một điểm ban đầu và di chuyển theo hướng của gradient (đạo hàm) của hàm mục tiêu để tìm đến điểm cực tiểu.

2 Công thức của Gradient Descent

Để thực hiện Gradient Descent, chúng ta sẽ cần tính toán đạo hàm của hàm mục tiêu $f(x)$:

$$f'(x)$$

Sau đó, chúng ta cập nhật giá trị của x theo công thức:

$$x_{\text{new}} = x_{\text{old}} - \eta f'(x_{\text{old}})$$

Ở đây:

- x_{new} : Giá trị mới của x sau mỗi lần lặp.
- x_{old} : Giá trị hiện tại của x .
- η : Tốc độ học (learning rate), là một tham số xác định bước nhảy trong từng lần cập nhật.

3 Lựa chọn Tốc độ học η

Tốc độ học η là một tham số rất quan trọng. Nếu η quá lớn, thuật toán có thể vượt qua điểm cực tiểu và dao động mà không hội tụ. Nếu η quá nhỏ, thuật toán sẽ hội tụ rất chậm và có thể không đạt được kết quả tốt trong thời gian hợp lý.

4 Thuật toán Gradient Descent

Dưới đây là các bước cơ bản để thực hiện Gradient Descent cho hàm một biến:

1. **Khởi tạo:** Chọn giá trị khởi điểm x_0 và tốc độ học η .

2. Lặp lại:

- Tính đạo hàm $f'(x_k)$.
- Cập nhật $x_{k+1} = x_k - \eta f'(x_k)$.
- Dừng lặp nếu đạo hàm $|f'(x_k)|$ nhỏ hơn một ngưỡng nào đó (tức là gần đến cực tiểu) hoặc số lần lặp đạt đến một giới hạn nào đó.

5 Ví dụ Minh Họa

Giả sử chúng ta có hàm:

$$f(x) = x^2$$

Đạo hàm của hàm này là:

$$f'(x) = 2x$$

Áp dụng Gradient Descent với $\eta = 0.1$:

1. Khởi tạo $x_0 = 1$.
2. Lặp lại cho đến khi hội tụ:
 - Tính $f'(1) = 2 \times 1 = 2$.
 - Cập nhật $x_1 = 1 - 0.1 \times 2 = 0.8$.
 - Tiếp tục lặp lại với x_1 , tính đạo hàm và cập nhật giá trị x .

Kết quả cuối cùng của chúng ta sẽ là một giá trị tiến gần đến 0, là điểm cực tiểu của hàm $f(x)$.

6 Kết luận

Gradient Descent là một công cụ mạnh mẽ giúp chúng ta tìm giá trị cực tiểu (hoặc cực đại) của các hàm mục tiêu. Đối với hàm một biến, quy trình này rất dễ hiểu và thực hiện. Tuy nhiên, quan trọng nhất là lựa chọn đúng tốc độ học η để đảm bảo thuật toán hội tụ nhanh chóng mà vẫn chính xác.

Dưới đây là ví dụ về cách viết mã Python để minh họa cho thuật toán Gradient Descent để tối ưu hóa hàm một biến:

```
import numpy as np
import matplotlib.pyplot as plt

# Định nghĩa hàm mục tiêu (là 1 hàm 1 biến)
def f(x):
    return x**2 + 4*x + 4

# Định nghĩa đạo hàm của hàm mục tiêu
def df(x):
    return 2*x + 4

# Hàm Gradient Descent
```

```

def gradient_descent(starting_point, learning_rate, num_iterations):
    x = starting_point
    history = [x]

    for i in range(num_iterations):
        gradient = df(x)
        x = x - learning_rate * gradient
        history.append(x)
        print(f"Iteration {i+1}: x = {x}, f(x) = {f(x)}")

    return history

# Thông số đầu vào cho hàm gradient descent
starting_point = 10      # Điểm khởi đầu
learning_rate = 0.1      # Tốc độ học
num_iterations = 20      # Số lần lặp

# Chạy gradient descent
history = gradient_descent(starting_point, learning_rate,
                             num_iterations)

# Vẽ đồ thị của hàm mục tiêu và quỹ đạo tìm kiếm
x = np.linspace(-10, 10, 400)
y = f(x)

plt.figure(figsize=(8,5))
plt.plot(x, y, label='hàm f(x)')
x_history = np.array(history)
y_history = f(x_history)

# Vẽ đường tìm kiếm của Gradient Descent
plt.plot(x_history, y_history, 'ro-', label="Gradient Descent Path")

plt.title('Gradient Descent Optimization')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.show()

```

6.1 Giải thích mã

1. Hàm mục tiêu và đạo hàm của nó:

- $f(x)$ là hàm mục tiêu cần tối ưu hóa.
- $df(x)$ là đạo hàm của hàm mục tiêu $f(x)$.

2. Hàm `gradient_descent`:

- `starting_point` là giá trị khởi đầu của x .

- `learning_rate` là tốc độ học.
- `num_iterations` là số lần lặp cần thực hiện.
- Trong vòng lặp, chúng ta cập nhật giá trị của x bằng cách di chuyển ngược theo hướng gradient.

3. Tham số đầu vào và Chạy hàm `gradient_descent`:

- Xác định điểm khởi đầu, tốc độ học, và số lần lặp.
- Thực hiện gọi hàm `gradient_descent`.

4. Vẽ đồ thị:

- Vẽ đồ thị hàm $f(x)$ và quỹ đạo của điểm x qua từng bước của thuật toán Gradient Descent.

Mã này cho phép bạn nhìn thấy quá trình tối ưu hóa của thuật toán Gradient Descent trên một hàm mục tiêu đơn giản.

Phần II

Gradient Descent cho hàm nhiều biến

7 Khái niệm cơ bản

Gradient Descent (Tiếng Việt: Hạ dốc theo Gradient) là một thuật toán tối ưu số cơ bản thường được sử dụng trong học máy và thống kê để tìm giá trị cực tiểu của một hàm mục tiêu. Khi làm việc với các hàm có nhiều biến số, việc hiểu và áp dụng Gradient Descent một cách chính xác là rất quan trọng. Trong bài viết này, chúng ta sẽ tìm hiểu cách Gradient Descent hoạt động trên hàm nhiều biến.

Gradient Descent là một phương pháp lặp lại mà chúng ta sử dụng để tối ưu hóa hàm mục tiêu. Mục tiêu của thuật toán là di chuyển từ một điểm khởi đầu ngẫu nhiên trong không gian tham số đến điểm cực tiểu của hàm giá trị. Với mỗi lần lặp, Gradient Descent điều chỉnh các tham số đi hướng về cực tiểu bằng cách di chuyển ngược lại hướng của vector gradient.

8 Công thức toán học của Gradient

Giả sử chúng ta có một hàm mục tiêu $J(\theta)$ mà chúng ta muốn tối ưu hóa, với θ là vector các tham số. Gradient của hàm $J(\theta)$ theo θ được tính như sau:

$$\nabla J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_n} \right)$$

Gradient là một vector chứa các đạo hàm riêng (partial derivatives) của hàm mục tiêu theo từng tham số θ_i .

9 Cập nhật tham số

Thuật toán Gradient Descent cập nhật các tham số θ theo công thức:

$$\theta := \theta - \alpha \nabla J(\theta)$$

Trong đó, α là hệ số học, còn gọi là tốc độ học (learning rate). Nó quyết định độ lớn của bước di chuyển trong không gian tham số. Nếu α quá lớn, thuật toán có thể không hội tụ. Nếu α quá nhỏ, thuật toán có thể hội tụ nhưng rất chậm.

10 Quy trình Gradient Descent

Gradient Descent được thực hiện qua các bước sau:

1. **Khởi tạo tham số:** Chọn ngẫu nhiên các giá trị ban đầu cho θ .
2. **Tính gradient:** Tính giá trị gradient $\nabla J(\theta)$ tại điểm hiện tại.
3. **Cập nhật tham số:** Điều chỉnh θ bằng cách di chuyển ngược lại hướng của gradient.
4. **Lặp lại:** Quay lại bước 2 cho đến khi hội tụ đạt được (gradient rất nhỏ hoặc số vòng lặp đạt tới giới hạn).

11 Ví dụ minh họa

Giả sử chúng ta có hàm mục tiêu:

$$J(\theta) = \theta_1^2 + \theta_2^2$$

Với gradient:

$$\nabla J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2} \right) = (2\theta_1, 2\theta_2)$$

Quá trình cập nhật tham số sẽ là:

$$\theta_1 := \theta_1 - \alpha \cdot 2\theta_1, \quad \theta_2 := \theta_2 - \alpha \cdot 2\theta_2$$

12 Những thách thức và biến thể

Khi áp dụng Gradient Descent cho hàm nhiều biến, có một số thách thức mà chúng ta cần phải lưu ý:

- **Chọn tốc độ học phù hợp:** Nếu chọn α không đúng, quá trình học có thể không hội tụ.
- **Thoát khỏi điểm cực tiểu cục bộ:** Hàm mục tiêu có thể có nhiều cực tiểu cục bộ. Gradient Descent có thể dừng lại ở một trong những điểm này thay vì cực tiểu toàn cục.

- **Gradient Descent với mini-batch:** Thay vì dùng toàn bộ dữ liệu, ta có thể chia dữ liệu thành các lô nhỏ (mini-batch) để cập nhật gradient, tăng tốc độ tính toán và ổn định hơn.

Các biến thể phổ biến của Gradient Descent bao gồm: **Stochastic Gradient Descent (SGD)** và **Mini-batch Gradient Descent**.

13 Kết luận

Gradient Descent là một công cụ mạnh mẽ để tối ưu hóa các hàm nhiều biến số trong học máy. Bằng cách hiểu rõ cách thức hoạt động và áp dụng đúng các kỹ thuật, ta có thể cải thiện hiệu suất của các mô hình và đạt được kết quả tối ưu hơn.

Hy vọng bài viết này đã giúp bạn hiểu rõ hơn về Gradient Descent cho hàm nhiều biến. Chúc bạn thành công trong các dự án học máy của mình!

14 Code lập trình

Chắc chắn rồi! Dưới đây là một ví dụ đơn giản về cách sử dụng Python để thực hiện thuật toán Gradient Descent trên một hàm nhiều biến. Trong ví dụ này, chúng ta sẽ tìm cực tiểu của hàm số $f(x, y) = (x - 3)^2 + (y - 2)^2$.

```
import numpy as np

# Định nghĩa hàm chi phí và gradient của nó
def f(x, y):
    return (x - 3)**2 + (y - 2)**2

def gradient(x, y):
    df_dx = 2 * (x - 3)
    df_dy = 2 * (y - 2)
    return np.array([df_dx, df_dy])

# Khởi tạo các tham số
x_init = 0.0      # Điểm bắt đầu x
y_init = 0.0      # Điểm bắt đầu y
learning_rate = 0.1  # Tốc độ học
num_iterations = 100  # Số lần lặp

x, y = x_init, y_init

# Vòng lặp Gradient Descent
for i in range(num_iterations):
    grad = gradient(x, y)
    x -= learning_rate * grad[0]
    y -= learning_rate * grad[1]
    if i % 10 == 0:  # In ra thông tin mỗi 10 lần lặp
        print(f"Iteration {i}: x = {x}, y = {y}, f(x, y) = {f(x, y)}")
```

```
print(f"Gia tri toi thieu cua ham f tai x = {x}, y = {y}, f(x, y)  
      = {f(x, y)}")
```

14.1 Giải thích

1. **Hàm chi phí và đạo hàm:** Hàm $f(x, y)$ là hàm chi phí mà chúng ta muốn tối thiểu hóa. Hàm $\text{gradient}(x, y)$ là gradient (đạo hàm) của $f(x, y)$.
2. **Khởi tạo:** Chúng ta khởi tạo các giá trị ban đầu cho x và y , tốc độ học (`learning_rate`), và số lần lặp (`num_iterations`).
3. **Vòng lặp Gradient Descent:** Trong vòng lặp, chúng ta cập nhật giá trị của x và y dựa trên gradient của hàm chi phí và tốc độ học. Chúng ta cũng in ra thông tin định kỳ để theo dõi quá trình học.
4. **Kết quả cuối cùng:** Sau khi hoàn thành số lần lặp, chúng ta in ra giá trị tối thiểu của hàm tại các giá trị x và y đã tìm được.

Chạy đoạn mã trên sẽ cho bạn thấy quá trình Gradient Descent tìm ra điểm cực tiểu của hàm số đã cho.